

Video Transcript

Video 4: More About Containers (06:49)

A development machine is a place of a lot of experimentation, and over time that machine can become unstable. You've changed too many configurations, you've installed too many packages, and ultimately you just need to reformat that drive and rebuild that machine, and that's a lot of work. You need to install your operating system, which can take a while to transfer. Put in all the patches, add all of your dependencies for runtimes, languages, and so on. You need to add your data stores, add your application servers and so this can take quite a bit. It can be frustrating; it can really sour your day. Now, when it comes to virtual machines, they really helped the situation quite a bit.

They gave us a foundation upon which we could add additional machines on top of our machine, and this was great for testing. In many ways, they got you closer to what you wanted, although you ultimately needed to add a few more things with most of the stacks that you were working on. However, there is a more fundamental question, and the question is can I run your code? And this turns out to be a difficult question to answer. You're ultimately replicating somebody else's environment. And so as it turns out, one of the frequent responses is your code will not work on my machine.

And one of the things that we've discovered is that shipping code to a server and getting it to run can be hard. And the reason for that is that our software has become more complex. We used to think of the LAMP stack, that is Linux, Apache, MySQL, and PHP or Perl. However, even just looking at the web stack, the picture has become much more complex. And although it might seem surprising, this is a simplification and so it's this next picture. The picture keeps on growing when it comes to hardware, that's true as well. We used to have one type of back end.

Now we have a multitude of devices that we're targeting as well as generalized platforms for computation. So, over time we have more complex software, we have more complex hardware, a varying number of these. And so a group that started to look at this problem created a picture similar to the one that I'm showing you here, where they identified a varying number of software packages even within those, a number of versions and a number of hardware platforms to target

as well. And so, what you have is this really challenging scenario where you are hard pressed to be able to replicate somebody else's environment consistently across all of this variability.

And so, as they thought about this problem and considered the history, they thought of shipping.

And you can see here that at one time, being able to ship something meant a varying number of sizes and weights and restrictions, as well as a varying number of ways in which you could ship it. This wasn't an easy problem. The solution that came to this world was the shipping container. And overnight, the world standardized on this box and was able to ship things around the world. And today if you go to any of the port cities, you will see these great stacks of the goods that are being moved around the world.

And so, a similar idea was conceived for software, one where you could package neatly your code in the environment that was provided. That was an extension of the work that had been done in Linux containers but was improved to a degree that it could be standardized and used as an industry abstraction to be able to replicate somebody's environment and run that code. And if you're curious about more of the history and the tremendous work that was done, you can look at this talk that reviews or goes over the history of this development. But ultimately the fundamental difference was that as opposed to loading a whole new machine on top of your machine, as was the case with virtual machines, containers leveraged the fact that you already had an operating system on your machine and simply added the additional executables and libraries, as you can see here, on top of a Docker runtime. So, this had great efficiencies. Not only that, there was a nice clean abstraction to be able to piecemeal, build new, new stacks of technology or new stacks of software. And you can see here, the idea above that base image of how you could add in the case of node if you wanted to, a node and then on top of that be able to use something else like Mongo.

And so, what was done with the success of this abstraction is that the industry adopted it in large numbers and a registry of images was built where today you can navigate to and see some of the packages that are popular being downloaded in the tens of millions and having a very productive ecosystem. More than that, you can define the application that you have built in a very compact text file. As you can see here, in 12 lines of code, I have defined a software application, which is a known application. In these 12 lines of code, I have provided everything that's needed for somebody else to recreate my environment, the operating system, the runtime, which in this case will be node, the code that is required, and once I publish this to the registry, they can pull it

with a command and be able to recreate my application. More than that, with extensions such as Docker Compose, you can stitch together an application involving several images like you can see here a database in addition to an application server. So, this was a great success. It has become one of the building blocks in the commercial clouds and companies like Google internally move billions of containers per week. And it has now become a building block, as I mentioned, to the commercial clouds, independent of which one you're using.

